

ADDITIONAL TOOLS & RESOURCES

FREE COMPUTING PLATFORMS

- *GitHub Codespaces* has **VS Code** as its GUI
 - native for Python and **GitHub Copilot**
 - provided by **GitHub**, a home of numerous *open-source* packages
- *Lightning.ai* has **VS Code** and **JupyterLab** as its GUIs
 - The VS Code interface has **GitHub Copilot**
 - **75 hours/month** free **GPU** access

LIGHTNING.AI

Free

Current

\$0

Students, researchers, hobbyists

Current plan

15 monthly credits

Key features

- ✓ 1 free active Studio (4 hour restarts)
- ✓ 32 core CPU Studios
- ✓ Single GPUs (L40S, A100, H100, H200...)
- ✓ 80% off on interruptible (spot)
- ✓ Up to 2 concurrent GPUs
- ✓ Persistent storage (50 GB limit)

- ✓ 15 req/min to our model APIs
- ✓ 120,000 tok/min to our model APIs
- ✓ Connect any local IDE or SSH
- ✓ Unlimited background execution
- ✓ Multiplayer live collaboration
- ✓ Use private and public models
- ✓ Access optimized Studios
- ✓ Automate with our SDK
- ✓ Pay as you go

- How much can a GPU accelerate?
 - Let's look at an application example

BIGRAM CLOUD

- **Word cloud**
 - visual representation of text data with
 - each word's **size** indicating its **frequency/importance**
 - *Bigram cloud* extends the concept to pairs of words (**bigrams**) instead of single words
- Application example:
 - What **risk factors** are mentioned in *Item 1A* of *Magnificent 7's* 10-K filings (2015 vs. 2025)?
 - AAPLE, MSFT, GOOGL, AMZN, NVDA, META, and TSLA

IDENTIFYING BIGRAMS IN TEXT

- **Stop words** (e.g., “of”, “the”, “is”, “and”)
 - removed to focus on more meaningful content
- Older approaches focused on **stemmed words**
 - reduced to their *root form*
(e.g., “vis” for vision, visions, visionary, and visibility)
- Newer approaches use **lemmatized words**
 - reduced to their *base form*
(e.g., “vision” for vision and visions; “visionary” for visionary)
- Lemmatization is computationally intensive, which benefits greatly from **GPU acceleration**

STUDIO ON LIGHTNING.AI

- **Studio** in Teamspace on *Lightning.ai*
 - is like **Codespace** on GitHub
- See the shared Studio template: **BigramCloud_GPUorCPU**

FREE COMPUTING PLATFORMS

- *GitHub Codespaces* has **VS Code** as its GUI
 - native for Python and **GitHub Copilot**
 - provided by **GitHub**, a home of numerous *open-source* codes
- *Lightning.ai* has **VS Code** and **JupyterLab** as its GUIs
 - The VS Code interface has **GitHub Copilot**
 - 75 hours/month free **GPU** access
- *Posit cloud* has **RStudio** (for **R**) as its GUI
 - free-tier users can use **GitHub Copilot**
 - but only for inline queries and code generation in the editor
 - What is **R**?

STORY BEHIND JUPYTER NOTEBOOK

- *Jupyter* notebook was introduced in Week 1 to kick off this module
- Embedded in the name **Jupyter** are letters representing
 - three major data science programming languages:
 - **Python**, **R**, and **Julia**
- **Julia** is a high-performance language a bit too advanced for this module
 - but **R** since its **tidyverse** era is very approachable

R VS PYTHON

- Python
 - a general-purpose programming language widely used for data analysis and **machine learning (ML)**
- R
 - a programming language specially designed for **statistical computing** and data analysis
- Their capability can be significantly expanded through importing
 - **libraries** (aka. **packages**) specialized in performing certain functions with commands outside the base language

R AND PYTHON

- Today, the data science related functionalities of the two languages have become very similar
 - Can often find similar libraries for both languages
 - e.g., **dplyr** for R and **pandas** for Python are powerful libraries for working with dataframes
 - Can *call R from Python* using the **rpy2** library
 - can also *call Python from R* using the **reticulate** package

SHELL VS GUI

- Code development in Python/R is often done in a *click-and-select* interface, i.e., **graphical user interface (GUI)**
- A **shell / terminal** lets you
 - interact with the computing environment using a **command line interface (cli)**
- A cli is not as user-friendly as a GUI
 - but can chain code in script files together for ***automated/batch processing***

FILE FORMATS OF CODE

`myscript` below refers to a script file containing programming code

- **Python:**
 - `.ipynb`: Python code in **Jupyter notebook** format
 - convenient for code development (not for shell use)
 - `.py`: Python code in script format (or marimo notebook as a script)
 - command to execute in shell: `python3 myscript.py`
 - **marimo** notebook offers the *best of both worlds*
- **R:**
 - `.rmd`: R code in **Rmarkdown** notebook format
 - convenient for code development (not for shell use)
 - `.r`: R code in script format
 - command to execute in shell: `Rscript myscript.r`

RSTUDIO PROJECT ON POSIT CLOUD

- **Project** in Workspace on *Posit cloud*
 - is like **Codespace** on GitHub
- See the shared Project:
 - **AF1204 - Week 10: Result Tables Using R**

STRENGTHS OF R ECOSYSTEM

- R's syntax is often more *concise* and *expressive* for data manipulation and analysis tasks

```
WK10_ResultTables.Rmd
28
29 # Merge datasets
30 ```{r}
31 You, yesterday * add more Week 10 files
32 ESG_data_new <- ESG_data |>
33   mutate(date = as.character(date)) |>
34   left_join(ESGscore_TobinsQ, by = c("instrument" = "tic", "date" = "datadate")) |>
35   mutate(
36     at_pos = ifelse(at > 0, at, NA), # Replace non-positive 'at' values with NA
37     ceq_pos = ifelse(ceq > 0, ceq, NA), # Replace non-positive 'ceq' values with NA
38     q = (prcc_f * csho + dlts + dlc) / at_pos, # Tobin's Q calculation
39     ESGscore = esg_metric,
40     Firm_Size = log(at),
41     Leverage = (dlts + dlc) / ceq_pos,
42     Year = fyear,
43     Industry = sich,
44   ) |>
45   select(source, q, ESGscore, Firm_Size, Leverage, Year, Industry) |>
46   drop_na() # Drop any remaining rows with NA values in the selected columns
47
48 ESG_data_B <- ESG_data_new |>
49   filter(source == "Provider_B") |>
50   select(-source) # Remove 'source' column as it's no longer needed
51
52 ESG_data_A <- ESG_data_new |>
53   filter(source == "Provider_A") |>
54   select(-source)
55
```

```
WK10_ResultTables.py > M Merge and engineer features > df = (
Generate + Code + Markdown | Run All Clear All Outputs | Run stale ... Select Kerne
1 # Merge and engineer features
Generate + Code + Markdown
Stale markdown
1 |
2 df = (
3   esg_data
4   .assign(date=lambda x: x["date"].astype(str))
5   .merge(esg_tq, left_on=["instrument", "date"], right_on=["tic", "datadate"],
6         how="left")
7   .assign(
8     at_pos = lambda x: x["at"].where(x["at"] > 0),
9     ceq_pos = lambda x: x["ceq"].where(x["ceq"] > 0),
10    q = lambda x: (x["prcc_f"] * x["csho"] + x["dlts"] + x["dlc"]) / x
11    ["at_pos"],
12    ESGscore = lambda x: x["esg_metric"],
13    Firm_Size = lambda x: np.log(x["at"]),
14    Leverage = lambda x: (x["dlts"] + x["dlc"]) / x["ceq_pos"],
15    Year = lambda x: x["fyear"],
16    Industry = lambda x: x["sich"],
17  )
18 )
19 df_B = df[df["source"] == "Provider_B"].drop(columns="source").reset_index(drop=True)
20 df_A = df[df["source"] == "Provider_A"].drop(columns="source").reset_index(drop=True)
Stale Python (marimo)

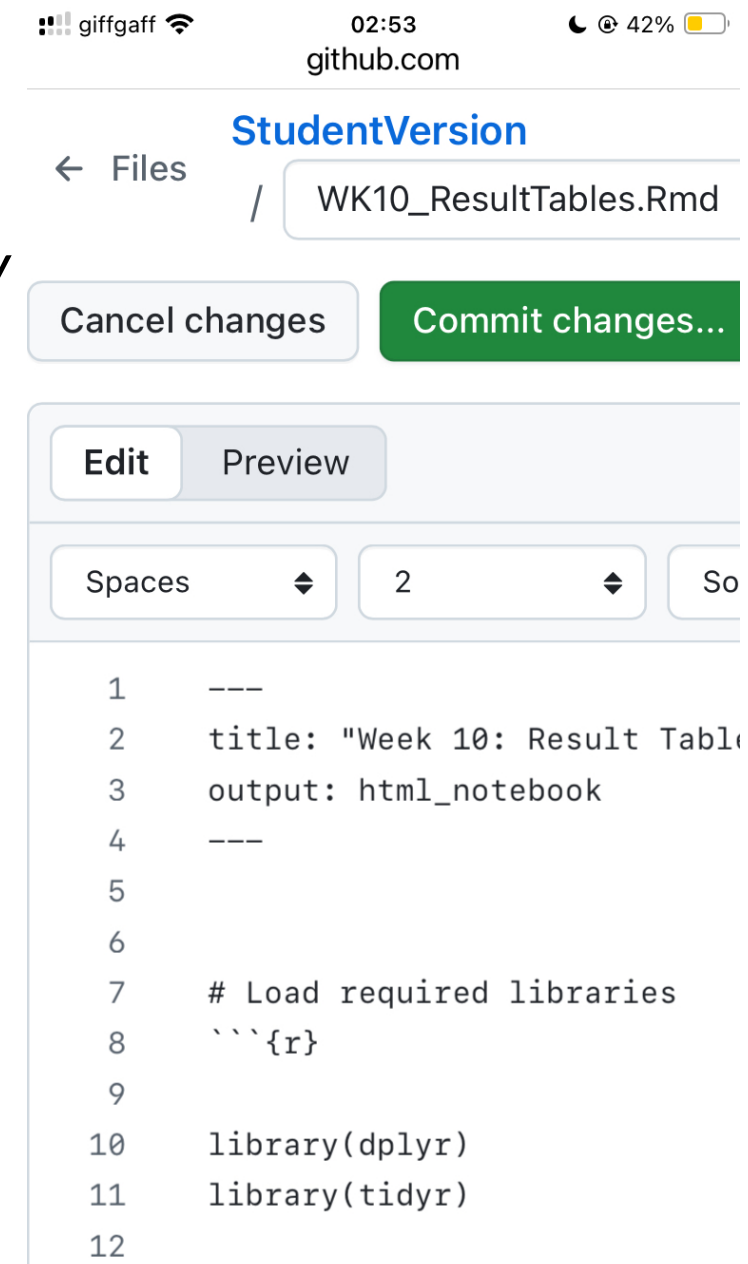
1 # Summary statistics
2 df_B.describe()
```

STRENGTHS OF R ECOSYSTEM

- Packages *specialized* for statistical analysis and econometrics are more likely to be available in R than Python
- For example,
 - *difference-in-differences* (DiD) analysis is an important methodology for measuring **causal effects**
 - This DiD website maintains a comprehensive list of packages for DiD analysis:
 - Over 15 packages in R vs. only 1 in Python

STRENGTHS OF R ECOSYSTEM

- **Rmarkdown** notebooks are **plain-text** files
 - easy to version-control
 - readable and editable even *on the go with only a phone*
- Structured in **JSON** format, **Jupyter** notebooks are
 - difficult to version-control effectively
 - **not** as *human-readable* or *edit-friendly* outside of the notebook interface
 - Similarly for **marimo** notebooks, despite being Python scripts (in plain text)
 - Next slide shows the **source code structures** of Jupyter and marimo notebooks



STRENGTHS OF R ECOSYSTEM

```
WK10_ResultTables.ipynb > ...
2 "cells": [
56 {
57   "cell_type": "code",
58   "execution_count": 8,
59   "metadata": {},
60   "outputs": [],
61   "source": [
62     "# Merge and engineer features\n",
63     "df = (\n",
64     "    esg_data\n",
65     "    .assign(date=lambda x: x[\"date\"].astype(str))\n",
66     "    .merge(esg_tq, left_on=[\"instrument\", \"date\"], right_on=[\"tic\", \"datadate\"],
67     "    how=\"left\")\n",
68     "    .assign(\n",
69     "        at_pos = lambda x: x[\"at\"].where(x[\"at\"] > 0),\n",
70     "        ceq_pos = lambda x: x[\"ceq\"].where(x[\"ceq\"] > 0),\n",
71     "        q = lambda x: (x[\"prcc_f\"] * x[\"csho\"] + x[\"dltt\"] + x[\"dlc\"] / x
72     "    [\n",
73     "        ESGscore = lambda x: x[\"esg_metric\"],\n",
74     "        Firm_Size = lambda x: np.log(x[\"at\"]),\n",
75     "        Leverage = lambda x: (x[\"dltt\"] + x[\"dlc\"] / x[\"ceq_pos\"],\n",
76     "        Year = lambda x: x[\"fyear\"],\n",
77     "        Industry = lambda x: x[\"sich\"],\n",
78     "    )\n",
79     "    [[\"source\", \"q\", \"ESGscore\", \"Firm_Size\", \"Leverage\", \"Year\", \"Industry\"]]\n",
80     "    .dropna()\n",
81     "    )\n",
82     "df_B = df[df[\"source\"] == \"Provider_B\"].drop(columns=\"source\").reset_index(drop=True)\n",
83     "df_A = df[df[\"source\"] == \"Provider_A\"].drop(columns=\"source\").reset_index(drop=True)
84 ]
85 {
86   "cell_type": "code",
87   "execution_count": 9,
```

Jupyter

```
WK10_ResultTables.py > ...
38
39
40 @app.cell
41 def _(esg_data, esg_tq, np):
42
43     df = (
44         esg_data
45         .assign(date=lambda x: x[\"date\"].astype(str))
46         .merge(esg_tq, left_on=[\"instrument\", \"date\"], right_on=[\"tic\", \"datadate\"], how=\"left\")
47         .assign(
48             at_pos = lambda x: x[\"at\"].where(x[\"at\"] > 0),
49             ceq_pos = lambda x: x[\"ceq\"].where(x[\"ceq\"] > 0),
50             q = lambda x: (x[\"prcc_f\"] * x[\"csho\"] + x[\"dltt\"] + x[\"dlc\"] / x[\"at_pos\"],
51             ESGscore = lambda x: x[\"esg_metric\"],
52             Firm_Size= lambda x: np.log(x[\"at\"]),
53             Leverage = lambda x: (x[\"dltt\"] + x[\"dlc\"] / x[\"ceq_pos\"],
54             Year = lambda x: x[\"fyear\"],
55             Industry = lambda x: x[\"sich\"],
56         )
57         [[\"source\", \"q\", \"ESGscore\", \"Firm_Size\", \"Leverage\", \"Year\", \"Industry\"]]\n",
58         .dropna()
59     )
60     df_B = df[df[\"source\"] == \"Provider_B\"].drop(columns=\"source\").reset_index(drop=True)
61     df_A = df[df[\"source\"] == \"Provider_A\"].drop(columns=\"source\").reset_index(drop=True)
62     return df_A, df_B
63
64
65 @app.cell
66 def _(df_B):
67     # Summary statistics
68     df_B.describe()
69     return
70
71
```

marimo

EXPANDING YOUR DIGITAL TOOLKIT

- No single tool solves every problem
 - Understanding the “right tool for the job” is a key skill for data professionals
- The landscape of Data Science and AI **changes rapidly**
 - Knowing where to find resources is essential for continuous **self-learning and exploration**
- For example,
 - see the newsletter [The Batch](#) and courses on [DeepLearning.AI](#)

ADOPTING OPEN-SOURCE RESOURCES

- **Availability** of open-source packages / data?
 - free vs. freemium vs. paid
- **Reliance** considerations:
 - Abandoned projects?
 - **Organizational backing** (e.g., **polars**) vs. Individual developer
 - **Active development**:
 - Last commit
 - Watch | Custom | Releases, Issues, Security alerts

ADOPTING OPEN-SOURCE RESOURCES

- Reliance considerations (cont'd):
 - **Maintenance** commitment:
 - Issues tab: Open vs. Closed
 - **Usefulness**
 - Stars, Used by, Contributors
 - **Star History** graph, **GitHub Trending** ranking
 - **Documentation** quality
 - README, docs/doc, Vignettes, Wiki, Website

THANK YOU !